Sentinel: In-Database Real-Time Fraud & Anomaly Engine

Course: CS/AI/AIM/DS 202 Database Management Systems (Semester III, A.Y. 2026-27)
Institution: ICFAI Foundation for Higher Education (IcfaiTech), Hyderabad **Authors:** Sai Haresh Anand S (25STUCHH010419) and Dulam Seshivardhini (25STUCHH001048)
Platform: MySQL 8.4 (Docker), pure SQL; no application code in the detection path

---

1. Abstract

Fraud checks are normally written in the application that accepts a payment. That arrangement has a structural weakness: any code path that writes to the database without going through the check, whether a batch job, an admin script or a second service, produces a transaction nobody scored. Sentinel moves the check into the database itself. Every `INSERT` into `transactions` fires an `AFTER INSERT` trigger that calls the stored procedure `score_transaction()`. The procedure evaluates six rules (amount Z-score, impossible travel by the Haversine formula, frequency velocity, unseen device, off-hours activity and merchant risk) against thresholds stored in the `rule_config` table, and writes a row to `alerts` with a JSON array explaining every rule that fired.

The schema has 13 tables in third normal form, 11 views, a user-defined function, four procedures, three triggers, a nightly event, four roles and a seeded data set of 12,067 transactions across 500 users and 597 accounts. Fund transfers run through `transfer_funds()`, which uses explicit row locks and a fixed lock order to stay atomic and deadlock-free. Measured on a laptop, the trigger raises the cost of an insert from about 0.04 ms to about 2 ms, which still sustains roughly 500 transactions per second. A date-range report falls from a full scan (cost 1,349) to an index range scan (cost 174) after a single-column index on `txn_time`. The project is verified by 102 SQL assertions and 27 Python integration tests.

2. Problem Statement and Objectives

2.1 Limitations of application-tier fraud checks

1. **Bypass.** The check runs only where a developer remembered to call it. Direct SQL, imports and a second service skip it.
2. **Race window.** If the application commits first and scores afterwards, there is an interval in which an unscored transaction is visible and spendable.
3. **Data movement.** Scoring needs history (an account's mean and deviation, its previous location, the last five minutes of activity). Pulling that history across the network for every payment adds latency and load, while the database already holds it next to an index.

4. **Hard-coded policy.** Thresholds live in source code, so changing one means a deployment.
5. **No audit trail by default.** Why an alert fired is usually a log line, not queryable data.

2.2 Objectives

#	Objective	How Sentinel meets it
O1	Score every transaction regardless of who inserted it	`AFTER INSERT` trigger on `transactions`
O2	Explain every alert	`alerts.reasons` JSON array, one entry per rule fired
O3	Make policy data, not code	`rule_config` holds every threshold and an enabled flag
O4	Keep money movement all-or-nothing	`transfer_funds()` with `START TRANSACTION`, `FOR UPDATE`, `COMMIT/ROLLBACK`
O5	Least-privilege access	Four roles; the reporting role cannot read raw transactions
O6	Measure the cost honestly	`EXPLAIN ANALYZE` plans and timed inserts, with limitations documented

3. Requirements and Entity-Relationship Design

3.1 Functional requirements

- Store users, their accounts, the devices and locations they transact from, and the merchants they pay.
- Record each transaction with amount, time, merchant, device, location and status.
- On each new transaction, decide whether it is suspicious and record why.
- Let an analyst move an alert through `open → investigated → confirmed | false_positive`, with the review time stamped and the change audited.
- Provide reporting views for dashboards without exposing base tables.
- Move funds between accounts atomically.

3.2 Entity sets

Table	Role	Primary key
`merchant_categories`	Lookup: category names	`category_id`
`locations`	Lookup: city, state, country, latitude, longitude	`location_id`
`devices`	Device type, OS, browser, fingerprint hash	`device_id`
`rule_config`	One row per fraud rule: enabled flag, threshold, parameters JSON	`rule_id`
`users`	Customer identity (name, unique email, phone)	`user_id`
`accounts`	Savings or current account owned by a user, with balance and status	`account_id`
`merchants`	Merchant with category, home coordinates, risk score	`merchant_id`
`account_devices`	Associative table for the many-to-many link between accounts and devices	(`account_id`, `device_id`)

Table	Role	Primary key
transactions	The fact table; 12,067 rows	txn_id
alerts	Output of the scoring engine	alert_id
behaviour_baselines	Per-account average, deviation, usual hours and cities	baseline_id
login_history	Account logins by device and location	login_id
audit_log	Who changed which row, with old and new values as JSON	audit_id

3.3 Relationships and foreign-key mapping

```
merchant_categories 1 ──< merchants 1 ──< transactions >── 1 accounts >── 1 users
                                                 |  \                          |
                                                 |   \ N:1 devices ──┤ M:N via account_devices
                                                 |    \ N:1 locations
                                                1
                                                 |
                                          alerts (1:1, UNIQUE txn_id)

accounts 1 ── 1 behaviour_baselines       accounts 1 ──< login_history >── devices, locations
```

Foreign key	References	On delete	Reasoning
accounts.user_id	users	CASCADE	An account has no meaning without its owner
merchants.category_id	merchant_categories	RESTRICT	A category in use cannot disappear
account_devices.account_id / device_id	accounts / devices	CASCADE	A link row dies with either side
transactions.account_id	accounts	CASCADE	History belongs to the account
transactions.merchant_id / device_id / location_id	merchants / devices / locations	RESTRICT	Referenced reference data must not vanish silently
alerts.txn_id	transactions	CASCADE	An alert explains one transaction
behaviour_baselines.account_id	accounts	CASCADE	Derived data follows its account
login_history.account_id	accounts	CASCADE	
login_history.device_id / location_id	devices / locations	RESTRICT	

Other declared constraints: UNIQUE on users.email, devices.fingerprint_hash, rule_config.rule_name, merchant_categories.category_name, (locations.city, locations.country), alerts.txn_id and behaviour_baselines.account_id; CHECK (amount > 0) on transactions; CHECK (balance >= 0) on accounts; range checks on latitude and longitude.

Design note. The schema was built week by week: Week 2 creates bare tables and Week 3 adds keys and constraints with ALTER TABLE. The result is identical to declaring everything inline; a schema fingerprint script compares it to a reference. The alerts table is created as fraud_alerts in Week 2 and renamed to alerts with its stray notes column dropped, which is why the design documents and the live database say alerts.

4. Normalised Relational Schema (1NF, 2NF, 3NF)

Method. For each table, list the functional dependencies (FDs) that hold in the business rules, determine the candidate keys, and check three conditions.

- **1NF:** every column is atomic and single-valued, and every row is identified by a key.
- **2NF:** no non-key attribute depends on a proper part of a composite key.
- **3NF:** for every non-trivial FD X → A, either X is a superkey or A is part of a candidate key. Equivalently, no non-key attribute depends on another non-key attribute.

4.1 Table-by-table verification

Table	Candidate keys	Non-trivial functional dependencies	Result
`users`	`user_id`, `email`	`user_id → full_name, email, phone, created_at`; `email → user_id`	Every determinant is a candidate key → **3NF (BCNF)**
`accounts`	`account_id`	`account_id → user_id, account_type, balance, status, created_at`	Single-column key rules out partial dependency (2NF). No non-key column determines another → **3NF**
`merchants`	`merchant_id`	`merchant_id → merchant_name, category_id, city, latitude, longitude, risk_score`	The category name is not stored here; it is reached through `category_id`, so `category_id → category_name` stays in `merchant_categories` → **3NF**
`transactions`	`txn_id`	`txn_id → account_id, merchant_id, device_id, location_id, amount, txn_time, status, metadata_json`	City, device type and merchant name are not repeated here; they live in the tables the foreign keys point to → **3NF**
`rule_config`	`rule_id`, `rule_name`	`rule_id → rule_name, enabled, threshold_value, parameters_json, updated_at`; `rule_name → rule_id`	Both determinants are candidate keys → **3NF (BCNF)**
`alerts`	`alert_id`, `txn_id`	`alert_id → txn_id, rule_name, risk_score, reasons, status, created_at, reviewed_at`; `txn_id → alert_id`	Both determinants are candidate keys → **3NF (BCNF)**
`account_devices`	(`account_id`, `device_id`)	(`account_id`, `device_id`) → `first_seen`, `last_seen`	The only composite-key table. Both columns describe the pairing, not either side alone, so there is no partial dependency → **2NF and 3NF**

The remaining six tables (`merchant_categories`, `locations`, `devices`, `behaviour_baselines`, `login_history`, `audit_log`) each have a single-column surrogate key and no dependency between non-key columns. In `locations`, `city → country` is not declared as a dependency because the same city name can occur in more than one country; the candidate key is (`city`, `country`) and `latitude`, `longitude` depend on it.

4.2 1NF in the presence of JSON columns

`alerts.reasons`, `rule_config.parameters_json`, `transactions.metadata_json` and the two JSON columns in `behaviour_baselines` hold structured documents. They are treated as atomic values: the engine never joins on or filters by an element inside them. `reasons` is output meant to be read whole by an analyst, and `parameters_json` carries free-form rule parameters. If the project later needed to query "all alerts that cite rule X", that would be the signal to extract a child table.

4.3 The deliberate exception

`behaviour_baselines` stores values that can be computed from `transactions` (mean, standard deviation, active hours, typical cities). It is a controlled cache rather than a normalisation error: the nightly procedure rebuilds it in full, so it cannot drift for more than a day, and it saves recomputing aggregates over an account's history for reporting.

5. Advanced Database Features

5.1 The six fraud rules

Each rule adds one point to the score and appends a human-readable string to the `reasons` array. A transaction is alerted if at least one rule fires. There is no combined threshold; each rule has its own, read from `rule_config` at scoring time.

#	Rule	Condition	Default threshold
1	**Amount Z-score**	`(amount − AVG) / STDDEV_SAMP` over the account's history exceeds the threshold; requires at least 5 prior-or-current rows and a non-zero deviation	3
2	**Travel velocity**	Haversine distance to the previous transaction is above 50 km and implied speed exceeds the limit	900 km/h
3	**Frequency velocity**	Number of transactions by the account inside the window, including the new one, exceeds the limit	more than 5 in 5 minutes
4	**New device**	The device is not in `account_devices` for this account	1 (enabled)
5	**Off-hours**	The hour of day accounts for less than 2% of the account's history	0.02
6	**Merchant risk**	The merchant has at least 3 transactions and at least 3 alerts, and its alert rate divided by the platform-wide alert rate exceeds the threshold	2 (min. alerts 3)

Haversine. Implemented as the deterministic function `haversine_km(lat1, lon1, lat2, lon2)` with Earth radius R = 6371 km:

```
a = sin²(Δφ/2) + cos φ1 · cos φ2 · sin²(Δλ/2)
d = 2R · asin(√a)
```

Hyderabad (17.385, 78.4867) to Tokyo (35.6762, 139.6503) evaluates to 6,311.39 km in MySQL. Fifteen minutes apart, that is an implied 25,246 km/h against a 900 km/h limit.

The 50 km floor stops two nearby cities with close timestamps from producing a spurious huge speed from small distances and short intervals.

A property of the Z-score rule worth stating. Because the trigger fires after the insert, the new row is already inside the average and standard deviation it is compared to. For a sample of n values, a single outlier cannot exceed a Z of $(n - 1)/\sqrt{n}$. With 65 transactions that ceiling is about 7.9. In practice this makes the rule slightly conservative for very small histories, and it is documented as a limitation rather than corrected, because computing the baseline from earlier rows only would require a second query per insert.

5.2 Scoring pipeline: `trg_after_txn_insert` and `score_transaction()`

```
CREATE TRIGGER trg_after_txn_insert
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    CALL score_transaction(NEW.txn_id);
END;
```

The trigger is `AFTER`, not `BEFORE`, because the procedure reads the stored row and a `BEFORE` trigger runs before the row exists. It is row-level, so a multi-row `INSERT` scores every row.

`score_transaction(p_txn_id)` loads the transaction and the matching `rule_config` rows, evaluates each enabled rule, builds the `reasons` array with `JSON_ARRAY` and, when the score is above zero, writes

```
INSERT INTO alerts (txn_id, rule_name, risk_score, reasons, status)
VALUES (..., 'combined', v_score, v_reasons, 'open')
ON DUPLICATE KEY UPDATE ...;
```

The `UNIQUE (txn_id)` key makes re-scoring idempotent. When the score is zero, any existing open alert for that transaction is deleted, so re-running the procedure after a threshold change leaves the table consistent. A bulk `backfill_alerts(first, last)` procedure applies the same logic to historical ranges.

A second pair of triggers handles the analyst workflow. `trg_before_alert_update` stamps `reviewed_at` when an alert leaves `open`. `trg_after_alert_update` writes the old and new status to `audit_log`, using `USER()` rather than `CURRENT_USER()`: inside a trigger `CURRENT_USER()` returns the definer, so every audit row would name root.

5.3 Baseline maintenance: `refresh_behaviour_baselines()`

```
CREATE PROCEDURE refresh_behaviour_baselines()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    DELETE FROM behaviour_baselines;
    INSERT INTO behaviour_baselines (account_id, avg_amount, stddev_amount,
                                     peak_hours_json, typical_cities_json)
    SELECT ... AVG(amount), STDDEV_SAMP(amount), JSON_ARRAYAGG(hour), JSON_ARRAYAGG(city)
    FROM transactions ... GROUP BY account_id;
```

```
    COMMIT;
END;
```

The delete and rebuild sit in one transaction with an exit handler, so a failure leaves the previous baselines intact instead of an empty table. The event `evt_refresh_baselines` calls it every day at 02:30 (the event scheduler must be on).

5.4 ACID transfers: `transfer_funds()`

The procedure moves money between two accounts and records the debit as a transaction, which the scoring trigger then evaluates like any other payment.

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

IF p_amount IS NULL OR p_amount <= 0 THEN SIGNAL SQLSTATE '45000' ...;
IF p_from_account = p_to_account      THEN SIGNAL SQLSTATE '45000' ...;

START TRANSACTION;

IF p_from_account < p_to_account THEN
    SELECT balance, status INTO v_from_balance, v_from_status
      FROM accounts WHERE account_id = p_from_account FOR UPDATE;
    SELECT status INTO v_to_status
      FROM accounts WHERE account_id = p_to_account   FOR UPDATE;
ELSE
    SELECT status INTO v_to_status
      FROM accounts WHERE account_id = p_to_account   FOR UPDATE;
    SELECT balance, status INTO v_from_balance, v_from_status
      FROM accounts WHERE account_id = p_from_account FOR UPDATE;
END IF;

-- missing / inactive account or insufficient funds: ROLLBACK, then SIGNAL

UPDATE accounts SET balance = balance - p_amount WHERE account_id = p_from_account;
UPDATE accounts SET balance = balance + p_amount WHERE account_id = p_to_account;
INSERT INTO transactions (account_id, merchant_id, device_id, location_id, amount, status)
VALUES (p_from_account, 1, p_device_id, p_location_id, p_amount, 'success');

COMMIT;
```

Property	Mechanism
Atomicity	Debit, credit and the transaction record commit together; the exit handler rolls back on any error, including one raised inside the scoring trigger
Consistency	Validation in the procedure, plus `CHECK (balance >= 0)` as a backstop
Isolation	`SELECT … FOR UPDATE` takes exclusive row locks, so a concurrent transfer on the same account waits instead of reading a stale balance
Durability	InnoDB redo log flushed at `COMMIT`

Deadlock avoidance. If transfer A→B and transfer B→A each locked their own source first, each would then wait for the other's lock forever. The procedure always locks the lower `account_id` first. All transfers acquire locks in the same global order, so a cycle of waits cannot form.

Scope limit. Only the sender's debit is recorded as a transaction, against placeholder merchant 1. Money received is not a purchase and would distort the receiver's spending statistics.

5.5 Indexing and performance

Indexes were created in Week 9 after the schema and the rule queries existed, so each one corresponds to a measured access path:

Index	Serves
`idx_txn_account_time (account_id, txn_time)`	Per-account velocity, previous-transaction lookup, baselines, latest-N queries
`idx_txn_merchant_time (merchant_id, txn_time)`	Merchant risk and merchant timelines
`idx_txn_time (txn_time)`	Date-range reports
`idx_txn_location (location_id)`	Location joins
`idx_alert_status (status, created_at)`	The open-alerts view
`idx_alert_rule (rule_name)`	Alerts by rule

Measured with `EXPLAIN ANALYZE` (MySQL 8.4.11).

Query	Before	After	Plan change
Daily report over a 6-day range (reads of 12,067 rows → 315)	cost **1,349**, table scan	cost **174**, range scan on `idx_txn_time`	about 8× cheaper
Latest 50 transactions of one account	cost 18.2 with filesort	cost 18.2, no sort	Same cost; the composite index removes the `Sort` step because rows come out already ordered
Merchant 50, newest 20	cost 88.5	reverse scan on `idx_txn_merchant_time`	no sort

The reduction from 1,349 to 174 therefore comes from the single-column index on `txn_time` serving a range predicate. The composite index on `(account_id, txn_time)` matters for a different reason: an account holds only about 52 rows on average, so the optimiser's cost does not drop, but the index lets the velocity and travel rules find an account's recent rows in order without sorting. We report both results as measured, and do not credit the composite index with the larger figure.

6. Experimental Benchmarks and Evaluation

Environment. MySQL 8.4.11 in Docker on a laptop under WSL2; single connection; data set of 12,067 transactions (500 users, 597 accounts, 50 merchants).

6.1 Trigger overhead

1,000 inserts were timed inside a transaction that was then rolled back, once with the trigger dropped and once with it present.

Configuration	Time for 1,000 inserts	Per row
No trigger	about 41 ms	about 0.04 ms
With `trg_after_txn_insert`	about 1,980 ms	about 2 ms

Scoring makes each insert roughly 50× slower in relative terms, and about 2 ms in absolute terms. That corresponds to approximately 500 scored transactions per second on one connection. For a payment path where a person is waiting on a card or UPI confirmation, 2 ms is negligible; for bulk loading it is not.

6.2 Bulk loading

Loading seed data with the trigger already installed stalled at 5,925 rows after 250 seconds, because every row ran the full scoring procedure against a growing table. The repository therefore loads data first and creates the trigger last (about 20 s for weeks 1–9), then uses `backfill_alerts()` to score history in one pass: `backfill_alerts(1, 12067)` takes about 38 s and produces **634 alerts, about 5%** of transactions.

6.3 Detection check

Twenty-seven planted persona transactions (ids 12041–12067) exercise each rule. They produce 21 alerts, and an ordinary purchase produces none. The database-side checks (102 assertions across 11 weeks) and the Python tests (27 tests) all pass at the time of writing.

6.4 Known limitations

- The seed data assigns random cities to transactions, so the travel rule fires 32–46 times on data that is synthetic rather than fraudulent.
- On amounts with a skewed distribution, a Z-score above 3 flags about 1.2% of rows (145 of 12,067). Amount is a poor fit for a normal-distribution test; a log transform would help.
- The off-hours rule is unreliable below about 50 transactions per account.
- Merchant risk feeds on alerts that other rules produce, so it can reinforce itself. The `min_alerts = 3` guard limits this but does not remove it.
- The baseline includes the row being scored (see 5.1).
- There is no user interface, and the code relies on MySQL-specific syntax.

7. Viva Questions and Discussion

Q1. What does the scoring trigger cost, and is it acceptable? It adds about 2 ms per inserted row in our measurement (about 0.04 ms without it), so a single connection scores about 500 transactions per second. For interactive payments that is far below human-visible latency, and the work is a few indexed lookups. It becomes a problem for bulk loads: loading our seed with the trigger present stalled, so we load first, create the trigger afterwards and use `backfill_alerts()` for history. Past a few thousand scored inserts per second one would move scoring to an asynchronous queue, which gives up the guarantee that scoring completes before the insert returns.

Q2. Why is the trigger `AFTER INSERT` and not `BEFORE`? `score_transaction()` looks the transaction up by id and aggregates over the account's rows including this one, so the row has to exist. A `BEFORE` trigger could modify `NEW` or reject the row, but it runs before the

row is stored. We use `BEFORE UPDATE` only where changing `NEW` is the point, to stamp `reviewed_at`. The consequence of `AFTER` is that the new row participates in the average and deviation, which caps the achievable Z-score near (n−1)/√n.

Q3. Explain the locking in `transfer_funds()`. How do you avoid deadlock? `SELECT … FOR UPDATE` inside a transaction takes an exclusive InnoDB row lock on the account, held until `COMMIT` or `ROLLBACK`. A second transfer touching the same account blocks, and when it resumes it reads the committed balance, so two transfers cannot both spend the same money. Deadlock occurs when two sessions hold one lock each and want the other's. We always lock the lower account id first, so A→B and B→A both take the same row first and the second simply waits. InnoDB would otherwise detect the cycle and abort one transaction; the lock order means that never has to happen.

Q4. What is the trade-off of keeping fraud logic in the database rather than in the application? Gains: it cannot be bypassed by any writer, there is no gap between commit and check, history is read next to its indexes, and thresholds are rows that an administrator can change without a release. Costs: the work consumes database CPU on the write path, the logic is MySQL-specific and harder to unit-test and debug than application code, a bug in a trigger can fail the originating insert, and scaling the database (usually the harder tier to scale) now also scales fraud scoring. Our compromise is to keep rules simple, indexed and set-based, and to keep model-style analysis outside the database.

Q5. Your report shows cost falling from 1,349 to 174. What caused it, and what does the composite index do? The 1,349 → 174 reduction is a date-range report on `transactions` moving from a full table scan of 12,067 rows to an index range scan of 315 rows on the single-column index `idx_txn_time`. The composite index `(account_id, txn_time)` follows the leftmost-prefix rule: it supports filters on `account_id` or on `account_id` plus a time range, and it returns an account's rows already sorted by time. In our measurement it did not lower the optimiser's cost for "latest 50 of an account" (18.2 either way, as an account has only about 52 rows) but it removed the sort step, which matters for the scoring rules. It does not help a filter on `txn_time` alone, which is why both indexes exist.

8. Conclusion and Future Scope

Sentinel shows that a useful fraud check can live entirely inside a relational database. A single trigger guarantees that every transaction is scored, whoever inserted it; thresholds are ordinary rows; every alert carries a machine-readable explanation; and money transfers keep ACID guarantees under concurrent access through a fixed lock order. The measured cost is about 2 ms per insert, and indexing the access paths reduced a representative report from cost 1,349 to 174. Equally important, the evaluation records where the design is weak: synthetic geography inflates the travel rule, the Z-score is a poor fit for skewed amounts, and the baseline includes the row under test.

Future work.

1. Calculate the baseline from earlier transactions only, removing the Z-score ceiling.

2. Assign each account a home city, so the travel rule compares against habitual locations and not just the previous row.
3. Score merchant risk from confirmed fraud (`alerts.status = 'confirmed'`) instead of from raw alerts, to break the feedback loop.
4. Replace the plain Z-score with a robust measure (median and MAD, or a log-amount Z-score).
5. Add a thin read-only API on the `api_reader` role and a dashboard on top of the existing reporting views.
6. Add a `txn_type` column so transfers, purchases and refunds are scored differently.
7. Test the asynchronous alternative (queue plus worker) to quantify the latency/throughput trade-off against the in-trigger design.

Figures in this report were measured on the project database; the supporting scripts and EXPLAIN output are in `week-09-views-indexes-dcl/docs/benchmark_measured.md` and `week-11-final-polish/docs/performance.md`.